

SECURITY RESEARCH MODULE

API SECURITY MASTERCLAS S

JWT // GRAPHQL // REST

AGENDA: EL CAMINO DEL ATACANTE

FASE 1

REST API & Logic Flaws

Análisis de superficies de ataque en arquitecturas REST, abuso de endpoints documentados y fallos en la lógica de autorización por objeto (BOLA) y función (BFLA).

FASE 2

JWT (JSON Web Tokens)

Deconstrucción de tokens, manipulación de algoritmos de firma, bypass de integridad y ataques dirigidos al almacenamiento y validación del servidor.

FASE 3

GraphQL Exploitation

Reconocimiento profundo mediante Introspección, bypass de autorización en resolvers, ataques de recursividad (DoS) y manipulación de queries para exfiltración masiva de datos.

LA API COMO SUPERFICIE CRÍTICA

Las APIs modernas han desplazado la lógica del servidor al cliente, exponiendo una superficie de ataque masiva que a menudo carece de los controles de seguridad tradicionales.

- **Interconectividad masiva:** Microservicios que confían ciegamente entre sí.
- **Shadow APIs:** Endpoints obsoletos (V1, V2) que permanecen activos sin monitoreo.
- **Exposición de Datos:** Respuestas JSON que incluyen campos sensibles filtrados por el frontend.



OWASP API TOP 10 (2023)

Categoría	Vulnerabilidad	Riesgo Principal
API1:2023	BOLA	Acceso a objetos de otros usuarios manipulando el ID.
API2:2023	Authentication	Mala implementación de JWT, tokens débiles o API Keys fijas.
API3:2023	Object Property Auth	Fuga de datos (Excessive Data Exposure) o Mass Assignment.
API5:2023	BFLA	Acceso a funciones administrativas desde cuentas regulares.
API6:2023	SSRF	Inyección de parámetros que obligan al servidor a pedir recursos externos.

Hoy nos enfocaremos en los vectores más críticos: **BOLA, Mass Assignment y Authentication (JWT)**.



MÓDULO 01: REST API TESTING

ENUMERACIÓN PASIVA DE ENDPOINTS

El primer paso es el reconocimiento. Antes de atacar, debemos mapear la superficie completa mediante fuentes que el desarrollador dejó expuestas.

Fuentes de Información

- Archivos JS: Análisis de rutas en el cliente (ej. main.js).
- Source Maps: Reconstrucción del código fuente original.
- Subdominios: APIs suelen vivir en api.* o dev.*.

Herramientas Clave

Burp Suite (Logger++), LinkFinder, Wayback Machine y motores de búsqueda especializados en APIs como **Shodan**.

FUGA DE DOCUMENTACIÓN TÉCNICA

Swagger (OpenAPI) es el estándar para documentar APIs. Si es accesible para el atacante, revela el esquema completo del backend.

Rutas Comunes

```
/api/v1/swagger.json  
/api/docs  
/v2/api-docs  
/swagger-ui.html
```

La documentación revela nombres de parámetros, tipos de datos y, crucialmente, **endpoints de administración**.

Impacto Crítico

Reduce el esfuerzo del atacante de "fuzzing" a cero.
Permite identificar métodos permitidos como DELETE o PATCH que no están en el frontend.

DEEP DIVE: BOLA (IDOR)

Ocurre cuando la aplicación valida que el usuario esté autenticado, pero **no valida** que tenga permiso sobre el objeto específico que está solicitando vía ID.

```
GET /api/user/101 → (Perfil de David: OK)  
GET /api/user/102 → (Perfil de Administrador: VULNERABLE)
```

La clave reside en el cambio de IDs predecibles (numéricos) o UUIDs exfiltrados de otras partes de la aplicación.

BOLA: ESTRATEGIAS DE DESCUBRIMIENTO

Manipulación de IDs

Probar con IDs incrementales, negativos o nulos. Probar a sustituir el ID de usuario por el ID de la organización.

Cruce de Roles

Intentar acceder a un objeto de Usuario A usando el token de Usuario B. Si el servidor responde con 200 OK, el control de acceso está roto.

Consejo: Buscar IDs no solo en la URL, sino en cabeceras como X-User-Id o en el cuerpo del JSON.

MECHANICS: BFLA

A diferencia de BOLA, BFLA afecta a la **funcionalidad**. El atacante accede a métodos protegidos que solo deberían ser para administradores.

```
GET /api/user/admin (403 Forbidden)
PATCH /api/user/admin (200 OK)
```

A menudo, el desarrollador protege el GET pero olvida el PUT o DELETE.

Vectores Comunes

- Cambio de verbos HTTP.
- Acceso a rutas /admin desde cuentas estándar.
- Sustitución de parámetros de rol en el body.

MASS ASSIGNMENT: CONCEPTO

Muchos frameworks mapean automáticamente el input del usuario (JSON/Form) a los modelos de la base de datos sin filtrar qué campos son editables.

Petición Legítima

```
{ "email": "david@qbit.io" }
```

El servidor solo espera el cambio de correo.

Ataque de Inyección

```
{ "email": "david@qbit.io", "role": "admin" }
```

Si el modelo tiene el campo role, el servidor lo actualizará.

MASS ASSIGNMENT: BUSINESS LOGIC

El impacto va más allá de los roles. Afecta directamente a la rentabilidad y seguridad de la lógica del negocio.

Contexto	Campo Inyectado	Consecuencia
E-Commerce	percentage_discount	Compra de productos por \$0 o con descuentos no autorizados.
Fintech	is_verified	Bypass de procesos de KYC (Know Your Customer).
SaaS	max_users	Uso ilimitado de licencias pagas de forma gratuita.

SSPP: TEORÍA DE LA CONTAMINACIÓN

Ocurre cuando el servidor concatena el input del usuario en una petición interna hacia otra API, permitiendo al atacante inyectar parámetros de control.

```
Petición Interna: /api/internal?user=[INPUT]&key=SECRET  
Atacante envía: admin&field=reset_token#
```

Cómo funciona

Usamos caracteres como # (%23) para truncar la petición interna o & para agregar parámetros que el backend procesará con mayor prioridad.

LAB 1: API DOCS & DELETE ABUSE

Objetivo

Identificar la superficie de ataque mediante el análisis de la documentación Swagger expuesta y utilizar métodos no autorizados para comprometer la integridad de la base de datos.

Resumen Técnico

Se intercepta la comunicación, se descubre el endpoint `/api/user` y se identifica que acepta el método DELETE. El atacante aprovecha la falta de validación de BFLA para eliminar al usuario objetivo.

LAB 2: MASS ASSIGNMENT & CHECKOUT

Resumen Teórico del Escenario

La aplicación expone un objeto JSON de carrito de compras durante el proceso de checkout. El atacante analiza la respuesta del servidor y detecta el campo `chosen_discount`.

Al inyectar manualmente el campo `"percentage": 100` en la petición POST final de compra, el servidor procesa el descuento malicioso, permitiendo adquirir la "Leather Jacket" por \$0.

```
Inyección: {"chosen_discount":{"percentage":100},"chosen_products":[{"product_id":"1","quantity":1}]}
```



MÓDULO 02: JWT SECURITY

ANATOMÍA DE UN JSON WEB TOKEN

Un JWT es un estándar (RFC 7519) para transmitir información de forma segura entre partes como un objeto JSON. Se compone de 3 partes separadas por puntos (.).

1. Header:

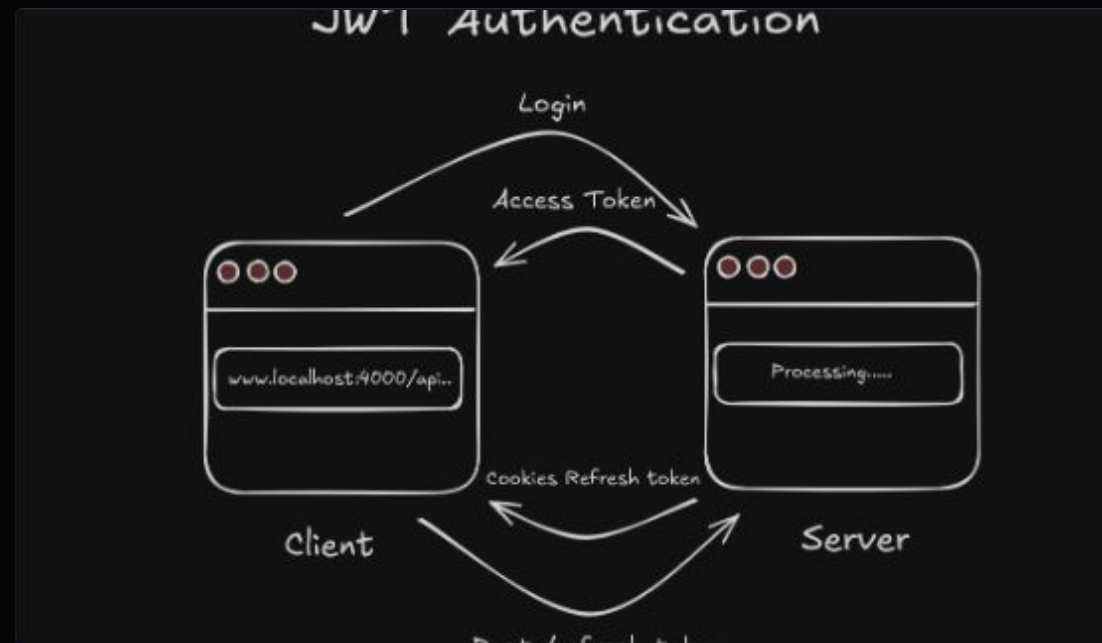
Define el algoritmo de firma (ej. HS256, RS256) y el tipo de token.

2. Payload:

Contiene los "claims" o datos (ej. usuario, expiración).

3. Signature:

Garantiza la integridad del token.



JWT: IDENTIDAD STATELESS

¿Por qué se usa?

Permite la escalabilidad horizontal. El servidor no necesita guardar sesiones en base de datos; simplemente verifica la firma criptográfica para confiar en el portador.

El Peligro del Lado del Cliente

Al guardarse a menudo en localStorage, el token es vulnerable a ataques de **XSS**. Si el atacante roba el token, roba la sesión completa.

| HMAC (HS256) VS RSA (RS256)

Existen dos formas principales de firmar un token. El error de configuración en el cambio entre ellas es un vector de ataque clásico.

Algoritmo	Tipo	Mecanismo
HS256 (HMAC)	Simétrico	Usa una única llave secreta para firmar y verificar. Si el secreto se filtra, todo el sistema cae.
RS256 (RSA)	Asimétrico	Usa una llave privada para firmar y una llave pública para verificar. Más seguro.

JWT: FIRMA NO VERIFICADA

Ocurre cuando la librería de JWT del servidor decodifica el token y lee el payload, pero **olvida llamar a la función de verificación** de la firma.

Explotación

El atacante simplemente modifica el payload (ej. cambia "sub": "david" por "sub": "admin") y envía el token. El servidor confía en el contenido sin validar si fue alterado.

ATTACK: EL ALGORITMO "NONE"

El estándar admite el algoritmo none para debugging. Si no se deshabilita en producción, el atacante puede forzar su uso.

```
Header: {"alg": "none", "typ": "JWT"}  
Payload: {"sub": "admin"}  
Signature: (Se elimina la firma)
```

Cómo se ejecuta

Se cambia el header, se edita el payload y se envía el token con un punto final vacío. Muchos servidores aceptarán este token "sin integridad".

ATAQUE: FUERZA BRUTA DE SECRETOS

Si se usa HS256, el secreto es la única barrera. Si es débil (ej. "123456", "secret", "admin"), puede ser crackeado offline.

```
$ hashcat -m 16500 jwt_token.txt -a 0 passwords.txt
```

Una vez obtenido el secreto, el atacante puede **forjar tokens válidos** para cualquier usuario del sistema indefinidamente.

JWT: KID INJECTION & PATH TRAVERSAL

Key ID (KID)

El header puede incluir un parámetro kid para indicar al servidor qué llave usar de un set. Si el servidor usa este campo en una query o ruta de archivo sin sanitizar...

- **Path Traversal:** Forzar al servidor a usar un archivo predecible como llave (ej. /dev/null).
- **SQL Injection:** Inyectar comandos SQL si el KID se consulta en la DB.

JKU (JWK SET URL) HIJACKING

El parámetro jku indica la URL donde se encuentra la llave pública de verificación. Si el servidor no valida el dominio de esta URL...

Explotación

El atacante aloja su propia llave pública en un servidor bajo su control, cambia el header del JWT para apuntar a su URL y el servidor verificará el token malicioso contra la llave del atacante.

LAB 3: SIGNATURE VALIDATION FAILURE

Resumen Teórico

Este laboratorio demuestra la confianza ciega del servidor en los claims del token. El atacante intercepta la petición al panel administrativo, modifica el valor de "sub" en el payload de "wiener" a "admin" y observa que el servidor concede privilegios sin validar la firma.



LAB 4: JWT ALG: NONE & ADMIN

Resumen Teórico

Se explora la vulnerabilidad de confusión de algoritmo. El atacante manipula el header del JWT para establecer el algoritmo a none, permitiendo el bypass total de la integridad criptográfica.

Al enviar el token modificado con el claim de "administrator", el backend acepta la identidad falsificada debido a una mala configuración en la lógica de validación del parser de JWT.



MÓDULO 03: GRAPHQL SECURITY

GRAPHQL: ARQUITECTURA DE CONSULTAS

A diferencia de REST, GraphQL es un lenguaje de consulta para APIs que permite al cliente solicitar **exactamente** los datos que necesita en una sola petición.

Schema:

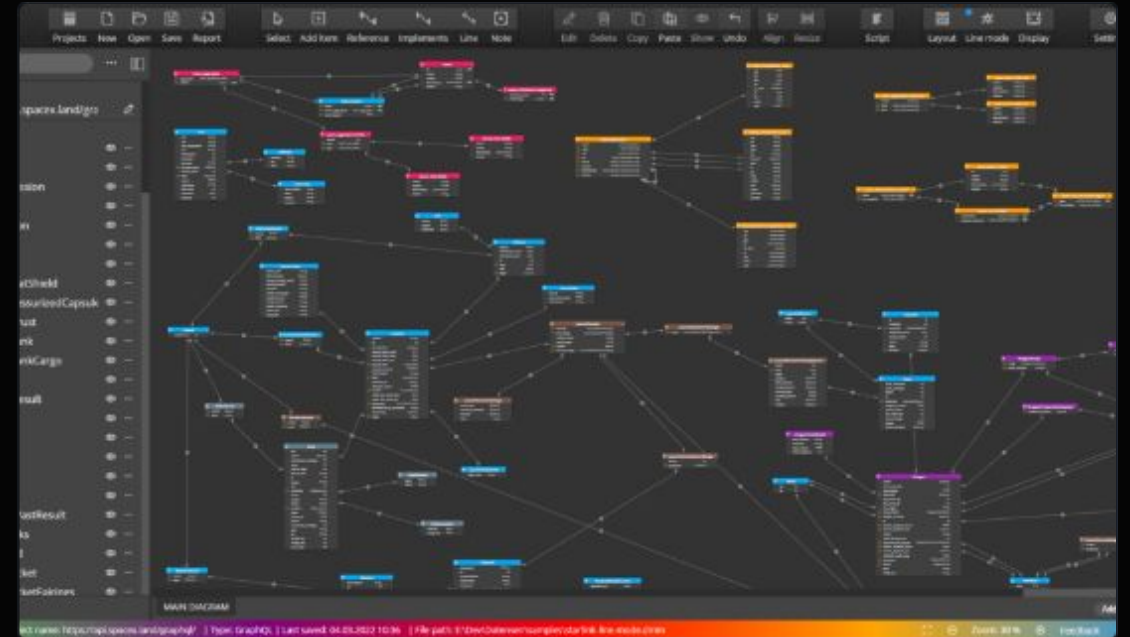
Define los tipos de datos y relaciones.

Queries:

Peticiones de lectura del cliente.

Mutations:

Peticiones de escritura/modificación.



GRAPHQL VS REST: POSTURA DE SEGURIDAD

Característica	REST	GraphQL
Endpoints	Muchos (vulnerable a BFLA)	Uno solo (vulnerable a DoS)
Autorización	A nivel de endpoint (URL)	A nivel de Resolver (Campo)
Reconocimiento	Swagger / Fuzzing	Introspección

INTROSPECCIÓN: EL MAPA DEL TESORO

Es una funcionalidad que permite consultar al servidor sobre su propio esquema. Si está activa en producción, regala toda la estructura al atacante.

```
query { __schema { types { name fields { name } } } }
```

Qué revela

Nombres de todas las tablas, todas las columnas, funciones de administración y campos ocultos (ej. password, isAdmin) que no se usan en el frontend.

BOLA EN GRAPHQL: FALLA EN RESOLVERS

En GraphQL, la autorización no se aplica a la URL, sino al **Resolver** que obtiene los datos de cada campo.

El Error

Muchos desarrolladores aplican auth en la query raíz pero olvidan aplicarla en los objetos anidados. Esto permite acceder a datos privados navegando por las relaciones del esquema.

```
query { user(id: 101) { orders { id secret_data } } }
```

QUERY COMPLEXITY: DOS VECTORS

Recursividad

Si el esquema permite relaciones circulares (ej. Autor → Libros → Autor), un atacante puede enviar una query infinita que agota la memoria del servidor.

Batching Abuse

Enviar miles de consultas pequeñas en un solo request HTTP para evadir sistemas de Rate Limiting basados en peticiones por segundo.

BYPASS: GRAPHQL BATCHING

Aprovechamos que GraphQL permite ejecutar múltiples queries en un solo array.

```
[{"query": "mutation { login(user: 'admin', pass: '123') }"}, {"query": "mutation { ... }"}]
```

Eficacia del Ataque

Podemos probar 500 combinaciones de password en una sola petición. El WAF verá 1 petición, pero el servidor ejecutará 500 intentos de fuerza bruta.

LAB 5: GRAPHQL INTROSPECTION & RECON

Resumen Teórico

El escenario plantea un servidor con la introspección habilitada. El atacante utiliza herramientas de mapeo para reconstruir el esquema completo, identificando el tipo User con un campo sensible llamado password.

Mediante una query dirigida, el atacante exfiltra las credenciales del administrador directamente, demostrando cómo una herramienta de desarrollo se convierte en un vector de fuga masiva.

LAB 6: GRAPHQL IDOR & DATA LEAK

Resumen Teórico Final

Se enfoca en la falta de autorización a nivel de resolver. El atacante manipula el argumento id en una consulta de perfil de usuario.

Al solicitar datos de un ID que no le pertenece, el servidor devuelve la información privada debido a que el código del resolver solo valida la sintaxis de la consulta y no la propiedad del recurso solicitado.

